



ADDING ROLLUP THREADS TO YOUR EXPERIENCE

Last Updated: 10/01/2018

If you've read [Using NDe APIs](#) and the [Product Feeds Developer's Guide](#), this guide provides the details necessary to integrate with the Product Feed Rollup Threads v2 API.

API at a Glance

Topic	Details
Use these APIs to	Display a product grid wall in a digital experience
Who calls this API?	Bootroom, Nike.com, Nike Running Club app (future)
Versions	v2
Supported Locales	See Language/Locale Mapping
SLA	Response time: 500ms Requests per second: 5 max (via edge router constraint)
Domain	Commerce
Prerequisites	API Registration
Contact Info	Slack: #nde-product-feeds Confluence: Product and Feeds API Product Owner: Andy Sun Apollo Product Owner: Patricia Cousins

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of [Using NDe APIs](#) on how to create and register your caller ID.

Authentication

No authentication or authorization is required to use this API.

Prerequisites

In order to use the Product Feed Rollup Threads v2 API, you need to:

- **Obtain a Consumer Channel ID – REQUIRED**

[Fill out a request form](#) to define your needs for a Consumer Channel ID. Once submitted, this form will be used to assess whether an existing ID can be used or a new ID needs to be created. This is different from the Channel ID you may be using to call the Product Feeds endpoint. See [Consumer Channel ID and Channel ID](#) for a comparison between the two ID types.

- **Configure Custom Search Rules – OPTIONAL**

The default key used for rolling up Threads is **productInfo.merchProduct.productRollup.key**. If you require any custom search rules to refine how Threads are rolled up, you can work with the Search team to create them. Contact the [Apollo Product Owner](#) for assistance.

Use Cases

I want to...	API(s) to use
Display a Product Grid Wall	Product Feed Rollup Threads v2 API

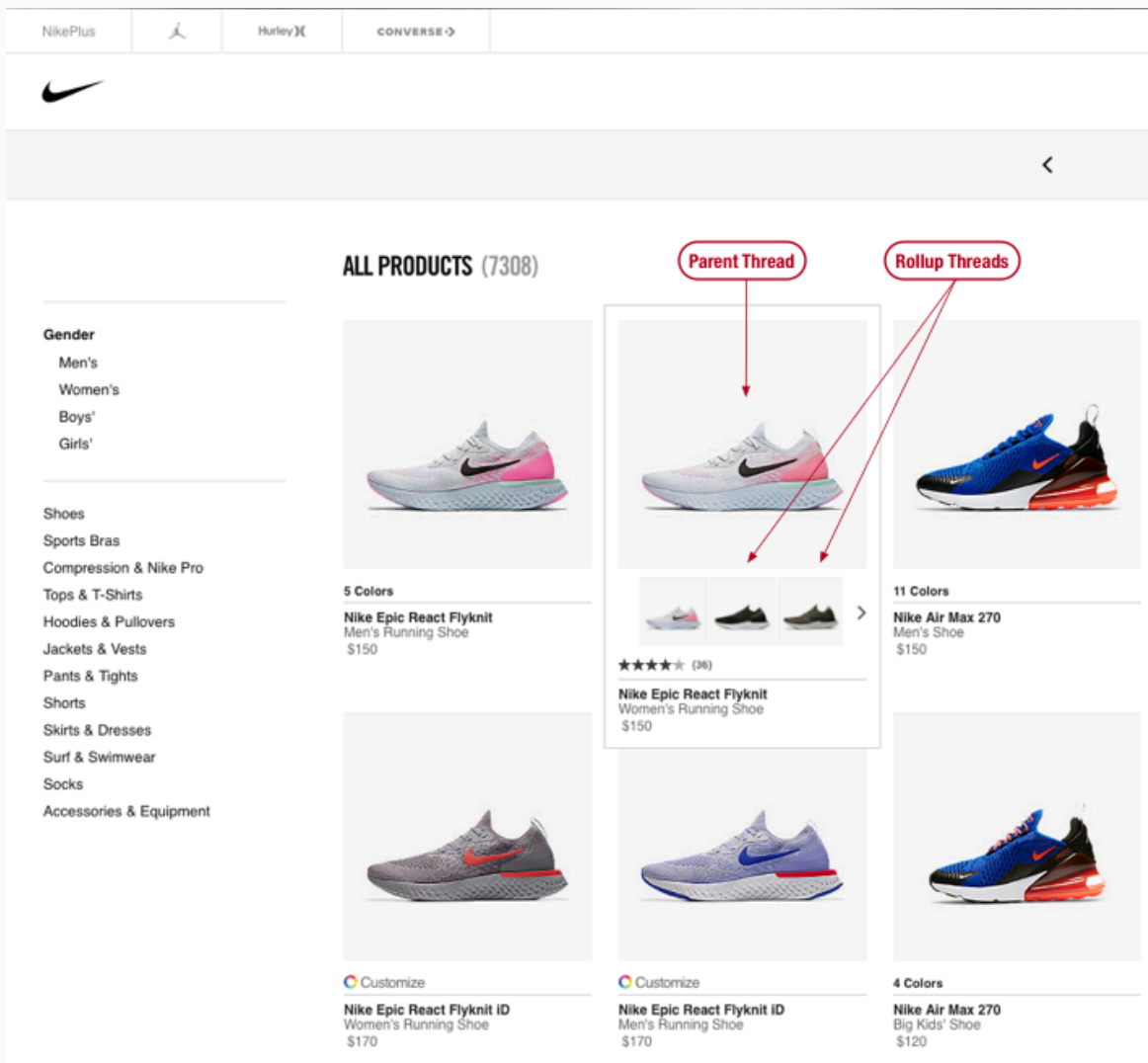
API Endpoint Quick Reference

HTTP Verb	Endpoint Name	Endpoint Description	URI Format
GET	Rollup Threads	Get a product Thread with related Threads nested within	/product_feed/ rollup_threads / v2{?filter,anchor,count,sort,searchTerms,rollupCount,rollupField,consumerChannelId}

What is a Rollup Thread?

The endpoint of this API is closely related to the *Threads List* endpoint of the Product Feeds v2 API. First, read the [Product Feeds v2 Developer's Guide](#) to understand the basic concepts of working with Threads.

Next, let's talk about what this API offers that is unique: Rollup Threads. A Rollup Thread is a Thread that is related to, and nested within a parent Thread. For example, there might be seven Rollup Threads nested within a given parent Thread, representing the other colors of a particular Nike shoe. Using Rollup Threads makes it much simpler for you to build a product grid wall experience like this:



How Can I Control the Rollup?

By default, the relationship between the Rollup Thread(s) to the parent Thread is based upon a rollup type and rollup key defined in the Nike's Prodigy product information system. As the client of the API, you do not have direct control over those values.

Both of these fields are located in the API response in the **productInfo.merchProduct.productRollup** object, field names **type** and **key**.

- rollup **type**: a group of products related by something, e.g. colors associated with a style

number, or shoes of the same width. Example values are “Standard”, “WidthGroup”, “NFL, or “NBA”.

- rollup **key**: a specific instance of a rollup type, e.g. a value representing a particular style number rollup. Example value is “xqTPKlqE”.

Custom Search Rules

You also have the ability to configure additional rules in the Apollo search administration tool that will **override the default rollup behavior described above, exclusively for your consumerChannelId**.

For example, if you wanted to rollup by something other than style number (i.e. “type”: “Standard”), you could define that as one or more rules in Apollo. You could also, for example, create a rule in Apollo to exclude customized Nike ID products or gift cards from your results, if desired.

Ultimately, Apollo is where you can control how the Rollup Threads are returned to you in the response from this API.

TIP: Reach out to the [Apollo Product Owner](#) for more information on how to use the Apollo tool.

Consumer Channel ID and Channel ID

Your Consumer Channel ID is unique to your app and allows you to have custom search rules to return only the parent and Rollup Threads that you need. But how is Consumer Channel ID related to the Channel ID you might be using with Product Feeds v2 API?

Consumer Channel ID and Channel ID are not directly related and are not used together in either API. They serve a similar purpose in that they are unique IDs that help you to get only the data you need from each API.

When calling the Product Feed Rollup Threads API, Consumer Channel ID is required and Channel ID is not allowed.

NOTE: Threads returned by the Product Feed Rollup
Threads API are pre-filtered for the Nike.com Channel ID.

I Already Use Product Feeds v2. How is This API Response Different?

The following diagram describes the how the structure of the response from the Rollup
Threads API differs from Product Feeds v2:



Rollup Threads API

Response Structure Compared to Feeds v2



Feeds v2: Threads List

GET /product_feed/threads/v2

```
{
  "pages":
  "objects": [
    {
      Thread 1
    },
    {
      Thread 2
    }
  ]
}
```

Rollup Threads List

GET /product_feed/rollup_threads/v2

```
{
  "pages":
  "objects": [
    {
      Thread 1 {
        "rollup": [
          {
            Related Thread A
          },
          {
            Related Thread B
          },
        ],
      },
      Thread 2 {
        "rollup": [
          {
            Related Thread X
          },
          {
            Related Thread Y
          },
        ],
      }
    }
  ]
}
```

"rollup" is an array of threads related to the parent thread

NOTE:

The structure of the responses has been simplified to highlight the differences. Refer to each API.md for the detailed schemas.

Using Rollup Threads

Use the Product Feed Rollup Threads v2 API to produce a grid wall of related products.

Rollup Threads List

Get a list of product Threads with related Rollup Threads using the *Rollup Threads List* endpoint.

Endpoint Details

HTTP Method	URI Path	JWT Restricted?
GET	/product_feed/ rollup_threads/ v2{?filter,anchor,co unt,sort,searchTerms ,rollupCount,rollupF ield,consumerChannel Id}	No

Path & Query Parameters

Parameter	Type	Description	Data Type	Required?
filter	Query	Restrict the response by one or more criteria, language, marketplace required	String	Required
consumerCha	Query	Consumer	String	Required

Parameter	Type	Description	Data Type	Required?
nnelId		channel ID of the experience (UUID)		
anchor	Query	Return elements after this anchor	String	Optional
count	Query	Maximum number of objects to return. Default: 10, Max: 60	String	Optional
sort	Query	Field(s) by which the results are sorted. Default: publishedContent.viewStartDateDesc , then id.keywordAsc .	String	Optional
rollupField	Query	Field by which to rollup the results. Default, and only supported field is productInfo.merchProduct.productRollup.key	String	Optional

Parameter	Type	Description	Data Type	Required?
rollupCount	Query	Count of items that will be inside the productRollup.rollups list. Default: 10, Max: 25	String	Optional
searchTerms	Query	Search for threads by one or more keywords separated with spaces. Request exact match by enclosing in double quotes. Default: partial match	String	Optional

Allowed Filter Parameters

The following is a list of scenarios that illustrate which **filter** parameters are supported:

TIP: Filter parameters **language** and **marketplace** are always required.

I Want to List	Sample Query
Threads for a taxonomy ID	https://api.nike.com/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-

I Want to List	Sample Query
	4976-858a-f159cf99c647&filter=marketplace(US)&filter=language(en)&filter=taxonomyIds(c2228131-f12b-4513-84cd-55ae15d6723d)

Allowed Sort Parameters

The following are the allowed fields that can be sent in the **sort** query parameter:

Sort Field	Description
publishedContent.publishStartDateAsc	By Content Publish Start Date, Ascending
publishedContent.publishStartDateDesc	By Content Publish Start Date, Descending
productInfo.merchProduct.commerceStartDateAsc	By Commerce Start Date, Ascending
productInfo.merchProduct.commerceStartDateDesc	By Commerce Start Date, Descending
productInfo.merchPrice.currentPriceAsc	By Current Price, Ascending
productInfo.merchPrice.currentPriceDesc	By Current Price, Descending
productInfo.merchProduct.commercePublishDateAsc	By Commerce Publish Date, Ascending
productInfo.merchProduct.commercePublishDateDesc	By Commerce Publish Date, Descending
effectiveStartSellDateAsc	By Effective Start Sell Date, Ascending
effectiveStartSellDateDesc	By Effective Start Sell Date, Descending
lastFetchTimeAsc	By Last Fetch Time, Ascending

Sort Field	Description
lastFetchTimeDesc	By Last Fetch Time, Descending

Using the Search Terms Parameter

Send one or more search keywords in the **searchTerms** query parameter to list only the threads that contain those keywords. In order for a thread to be returned in the response, all included keywords must be found in a searchable field within that thread.

The searchable fields are:

- `productInfo.productContent.fullTitle`
- `productInfo.productContent.title`
- `productInfo.productContent.subtitle`
- `publishedContent.properties.consumerLabels.classification.text`
- `productInfo.merchProduct.styleColor`
- `productInfo.merchProduct.styleCode`

Search Summaries are available when using the **searchTerms** parameter. If any search terms are corrected for spelling (or another reason), a **searchSummary** section will be added to the pages section of the response. Within that section, a field for **originalTerms** contains the original input, while a field for **correctedTerms** contains the corrections for the original terms.

For example, using `searchTerms=Chuck Taylor` would return any threads where the words 'Chuck' and 'Taylor' are found anywhere in a searchable field.

The default search behavior is *partial match*. Limiting the search to only *full string matches* can be done by enclosing the keywords in double quotes, like `searchTerms="Chuck Taylor"`. In this case, the thread must contain the exact full string 'Chuck Taylor' in a searchable field in order to be returned in the response.

Example Scenarios

Let's take a look at a few *Rollup Threads List* scenarios.

I Want to List	Sample Query
Search all on Nike.com products in US by “Men’s Jordan”, sorted by newest first	<code>https://api.nike.com/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-4976-858a-f159cf99c647&filter=language(en)&filter=marketplace(US)&searchTerms=Men’s%20Jordan&sort=effectiveStartSellDateDesc</code>
TBD	TBD

Request Headers

There are no required request headers.

Request Body

There is no request body required for a GET request.

Sample *Rollup Threads List* request URI:

```
https://api.nike.com/commerce/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-4976-858a-f159cf99c647&filter=language(en)&filter=marketplace(US)
```

Response Body

NOTE: The **productInfo** array contains responses from up to 8 other APIs, and are formatted according to the same schema as the source APIs. Links are provided to the relevant API.md for you to find the corresponding

response schema.

Element Name	Type	Description	Required?
pages	object	Object at top level containing link to previous and next pages of results	Required
pages.prev	string	Link to previous page of results	Required
pages.next	string	Link to next page of results	Required
pages.totalPages	integer	Total number of pages	Optional
pages.totalResources	integer	Total number of resources	Optional
pages.searchSummary	object	Context for the user's keyword query and what was decomposed from it	Optional
pages.searchSummary.originalTerms	string	The original search terms input by the user	Optional
pages.searchSummary.correctedTerms	array	A list of key/value pairs, mapping individual, incorrect search terms from the user with the correct ones	Optional

Element Name	Type	Description	Required?
pages.searchSummary. synonymTerms	object	A mapping of search terms that were successfully mapped to synonyms; the search term is the key, and the value is the list of synonyms	Optional
pages.searchSummary. concepts	array	A list of concepts that matched the user's searchTerms query	Optional
objects	object	Array at top level containing feed data	Required
objects. id	string	Unique identifier for the feed in UUID format	Required
objects. channelId	string	UUID for the channel (collectionGroupId)	Optional
objects. channelName	string	Human-readable name for the channel	Optional
objects. marketplace	string	ISO 3166 two-letter country code for the user's current location	Required
objects. language	string	BCP-47 language	Required

Element Name	Type	Description	Required?
		code	
objects. lastFetchTime	string	Time when the data was aggregated in ISO-8601 compliant format: yyyy-MM-ddTHH:mm:ss.SSSZZ	Required
objects. active	boolean	Indicator for whether or not this thread is currently available for general use	Optional
objects. publishedContent	object	API Reference	Required
objects. productInfo	array	Array of responses from other APIs with product info	Optional
objects.productInfo. merchProduct	object	API Reference	Required
objects.productInfo. merchPrice	object	API Reference	Required
objects.productInfo. skus	object	API Reference	Optional
objects.productInfo. availability	object	API Reference	Optional
objects.productInfo. availableSkus	object	API Reference	Optional
objects.productInfo	object	API Reference	Optional

Element Name	Type	Description	Required?
.productContent			
objects.productInfo .launchView	object	API Reference	Optional
objects.productInfo .imageUrls	object	Object containing product image URL	Optional
objects.productInfo .imageUrls. productImageUrl	string	URL for product image	Optional
objects.productInfo .customizedPreBuild	object	API Reference	Optional
objects. rollup	object	Object containing rollup info	Optional
objects.rollup. totalThreads	integer	The total threads available in the rollup response including the master thread	Optional
objects.rollup. threads	array	Array of threads that are related by a rollup key	Optional
objects. resourceType	string	Type of resource being returned in the response	Required
objects.links.self. ref	string	Self-link to this resource	Required

TIP: For sample responses, see the [Rollup Threads API Reference](#).

Upgrading to the Latest Version

Version 2 (v2) is the current and only version of this API.

Best Practices

See the Best Practices section of the [Product Feeds Developer's Guide](#).

Troubleshooting

I'm getting a 200 response, but the Thread data and/or Rollup Threads are not as expected

- Check your Smart Search rules configuration in the Apollo application to ensure that the rules are correct.
- Check the rollup key & type from Prodigy for the Parent Thread is as expected.
- Reach out to Product Feeds team on Slack for assistance: [#nde-product-feeds](#)

TIP: See the Troubleshooting section of the [Product Feeds Developer's Guide](#) for more general troubleshooting information.

Glossary

See the [Glossary](#).

Document Change Log

Summary	Date	Description
Initial draft	05/17/2018	Initial Draft
Update	07/19/2018	Updated how to obtain a consumerChannelId
Updated TOC	10/01/2018	Removed 'In this guide', replaced with sidebar TOC

Related Links

[NDe Docs Home](#)

[Get Started](#)